

FrontierAtlas Intelligence Graph — Production Architecture

AI Engineer Assessment · Scale Strategy, LLM Orchestration, Freshness & Hybrid Storage

1. Scale Strategy — Ingesting 500,000+ Records Autonomously

The ingestion engine is architected as a fleet of **stateless, horizontally-scalable async workers** rather than a monolithic sequential crawler. Ingestion is partitioned into discrete task units (e.g., Arxiv category offset slices, directory pagination URLs, company domain slugs) pushed onto a durable distributed work queue (Redis Streams / AWS SQS in production; an in-memory `asyncio.Queue` in this trial). Scaling from 1,000 to 500,000+ records requires zero code modifications—only increasing horizontal worker replicas or raising queue concurrency limits.

Decoupled Discovery & Extraction: Discovery workers (identifying new entity URLs, RSS feeds, and GitHub repos) operate on isolated queues from extraction workers (fetching content, running LLM parsing, and schema validation). This prevents upstream rate limits or slow HTML fetches from stalling downstream throughput.

Concurrency & Courtesy Controls: Downstream endpoints are protected via an `asyncio.Semaphore` and a token-bucket `RateLimiter` per domain/API. For structured repositories (Arxiv Atom & GitHub REST APIs), we leverage official programmatic endpoints rather than brittle HTML scraping, maximizing throughput while eliminating anti-bot friction.

2. Resilient LLM Orchestration — 413 & 429 Mitigation

429 Rate Limit Management (Exponential Backoff with Jitter): Every LLM extraction runs through `LLMOrchestrator._call_with_backoff`. When a 429 is encountered, the system applies exponential backoff ($\$_{delay} = \min(\text{base} \cdot 2^{\text{attempt}}, 60s)$) randomized with $\pm 50\%$ jitter to prevent thundering-herd synchronization across worker fleets. If max retries are exhausted, the orchestrator seamlessly fails over to the next provider in the chain.

Multi-Tier Provider Chain: Primary: Gemini Flash (gemini-3.6-flash, ~1M context) → Fallback: OpenAI ChatGPT (GPT-4o-mini, 128k context). Each tier returns structured JSON only, strictly validated against Pydantic schemas before persistence.

413 Payload Too Large (Semantic Chunking & Dynamic Halving): Raw text is pre-chunked via `chunk_text()` budgeted to each provider's context window. Chunking preserves paragraph and sentence boundaries with a 15% sliding overlap to maintain entity claim context. If a 413 is triggered, the chunk is dynamically halved and retried up to twice before failing over to the next tier.

3. Distributed Freshness & Deduplication (< 24h Window)

Real-time monitoring across distributed crawler nodes enforces a strict 24-hour freshness sliding window across AI Jobs and News verticals via two synchronized layers:

- **Timezone-Aware Date Normalization:** `src.utils.dates.normalize_date` parses diverse date representations (ISO-8601, RFC-2822, relative strings like '2 hours ago') into canonical UTC timestamps. Records are admitted into Jobs/News only if `is_within_last_24h(pub_date)` evaluates to True against synchronized NTP clocks.
- **Idempotent Distributed Deduplication:** Every scraped entity generates a deterministic dedupe key (canonical URL or `SHA256(source + title + pub_date)`). In production, this key is checked atomically against a centralized Redis Bloom Filter and committed via PostgreSQL `ON CONFLICT DO UPDATE`. This guarantees multiple workers encountering the same posting cannot create duplicate records.
- **Seen-Before Heuristic:** For sources lacking explicit publication dates, the `SeenBeforeHeuristic` state tracker admits records only if their hash was unseen in prior crawl cycles, strictly preventing the re-ingestion of stale data.

4. Storage Strategy — Hybrid Polyglot Architecture

FrontierAtlas employs a purpose-built polyglot persistence architecture combining relational, graph, vector, and object storage tiers:

Storage Tier	Technology	Role & Data Assets
Relational Core	PostgreSQL 16	Primary system of record. Stores structured entities (Startups, Products, Papers, Jobs, News, Resolution Logs). Uses indexed JSONB columns for flexible metadata with strict ACID transactional guarantees.
Graph Engine	Neo4j / Apache AGE	Models multi-hop relationships: (Startup)-[:PRODUCES]->(Product), (Startup)-[:PUBLISHES]->(Paper)-[:HAS_REPO]->(GitHub), enabling deep graph queries without recursive SQL joins.
Vector Layer	pgvector / Pinecone	Embeddings of paper abstracts and product descriptions. Powers semantic deduplication and entity resolution beyond string similarity.
Object Archive	AWS S3 / Cloudflare R2	Raw HTML and PDF snapshots stored with immutable content hashes. Ensures 100% auditability back to original source web pages, guaranteeing zero hallucinated data.

5. Deterministic Entity Resolution Engine

To prevent duplicate company records (e.g. 'OpenAI' vs. 'OpenAI, Inc.'), the 4-tier resolution engine applies: **1. Exact alias lookup** (canonical seed dictionary) → **2. Case & legal suffix stripping** (Inc, LLC, Corp) → **3. RapidFuzz token-sort matching** (threshold ≥ 0.87) → **4. Passthrough registration**. Every resolution emits an `EntityMappingLogRecord` with confidence scores and method provenance into `output/entity_mapping_log.csv`.